



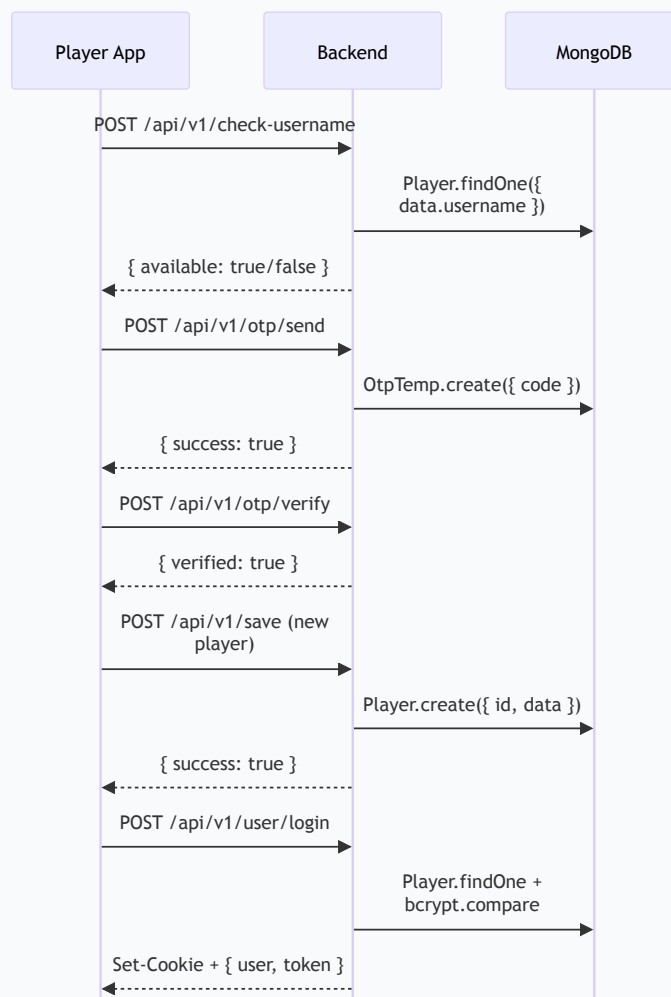
Player Architecture

Complete system flow from the mobile app player perspective

AceTrack Architecture Documentation • Generated 12 June 2026

1. Authentication Flow

The player's journey begins with authentication. Two paths exist: new user registration (OTP-verified) and existing user login (username/password).



Detailed Flow Breakdown

- 1 Username Check**
Player submits desired username. Backend checks MongoDB for case-insensitive match using regex query.
`POST /api/v1/check-username { username }`
- 2 OTP Dispatch**
System generates a 6-digit verification code, stores it in OtpTemp collection with expiry. Simulated SMS/email delivery for pre-production.
`POST /api/v1/otp/send { phone/email, purpose: "signup" }`

3 OTP Verification

Player submits received code. Server validates against OtpTemp record, checking expiry and max attempts. Rate-limited to prevent brute force.

```
POST /api/v1/otp/verify { phone/email, code }
```

4 Account Creation

New player record saved via the unified /save endpoint. Creates a Player document with hashed password (bcrypt), initial credits, and empty profile.

```
POST /api/v1/save { data: { players: [newPlayer] } }
```

5 Login (Returning Users)

Username/password authentication with bcrypt comparison. On success, issues JWT as HttpOnly cookie (ace_auth_token) for stateless session management.

```
POST /api/v1/user/login { username, password }
```

6 Session Verification

Every subsequent API call verifies the JWT cookie. The /auth/me endpoint returns current user data and role for client-side hydration.

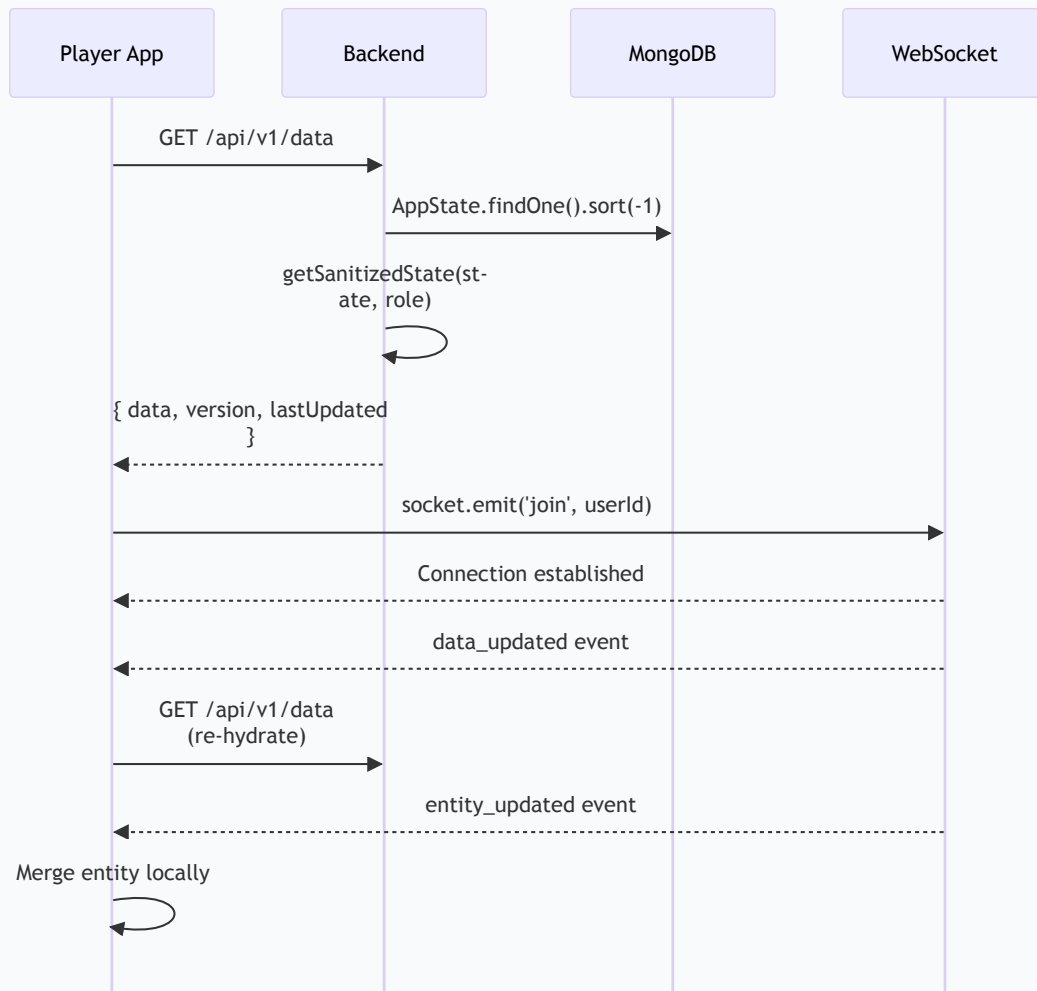
```
GET /api/v1/auth/me (Cookie: ace_auth_token)
```

API Endpoints

ENDPOINT	METHOD	AUTH	PURPOSE
/api/v1/user/login	POST	Rate limited	Player login with username/password
/api/v1/auth/me	GET	API Key	Session verification via HttpOnly cookie
/api/v1/check-username	POST	API Key	Check username availability
/api/v1/otp/send	POST	Rate limited	Send OTP for signup verification
/api/v1/otp/verify	POST	Rate limited	Verify OTP code
/api/v1/logout	POST	None	Clear auth cookie
/api/v1/auth/change-password	POST	API Key	Change password (authenticated)

2. State Hydration & Real-Time Sync

The core data pipeline that keeps the player app synchronized with the server using a combination of REST hydration and WebSocket push events.



Detailed Flow Breakdown

1 Initial Hydration

On app launch, fetches the full application state. The server retrieves the latest AppState document (sorted by version descending) and sanitizes it based on user role — stripping PII and admin-only fields for non-admin users.

```
GET /api/v1/data (x-ace-api-key + Cookie)
```

2 Role-Based Sanitization

The `getSanitizedState()` function filters the response. Players see only public tournament data, their own profile, and approved videos. Admin/support fields like audit logs, wallet history of others, and internal notes are removed.

3 WebSocket Connection

Player joins their personal Socket.IO room (`user:{userId}`). This enables targeted real-time updates and admin device pings.

4 Push-Based Sync (`data_updated`)

When any user saves data, the server broadcasts a `data_updated` event with the changed entity keys and new version number. The client re-fetches only if the version is newer than its local copy.

5 Entity-Level Sync (`entity_updated`)

For granular changes (tournament update, video status change), the server emits `entity_updated` with the specific entity type and data. The client merges this directly into local state without a full re-hydration.

6 Save Flow (Player Edits)

When a player edits their profile or performs actions, changes are sent via `POST /save`. The server acquires a sync mutex, performs delta detection to identify changed fields, and writes atomically to both the distinct collection and legacy AppState mirror.

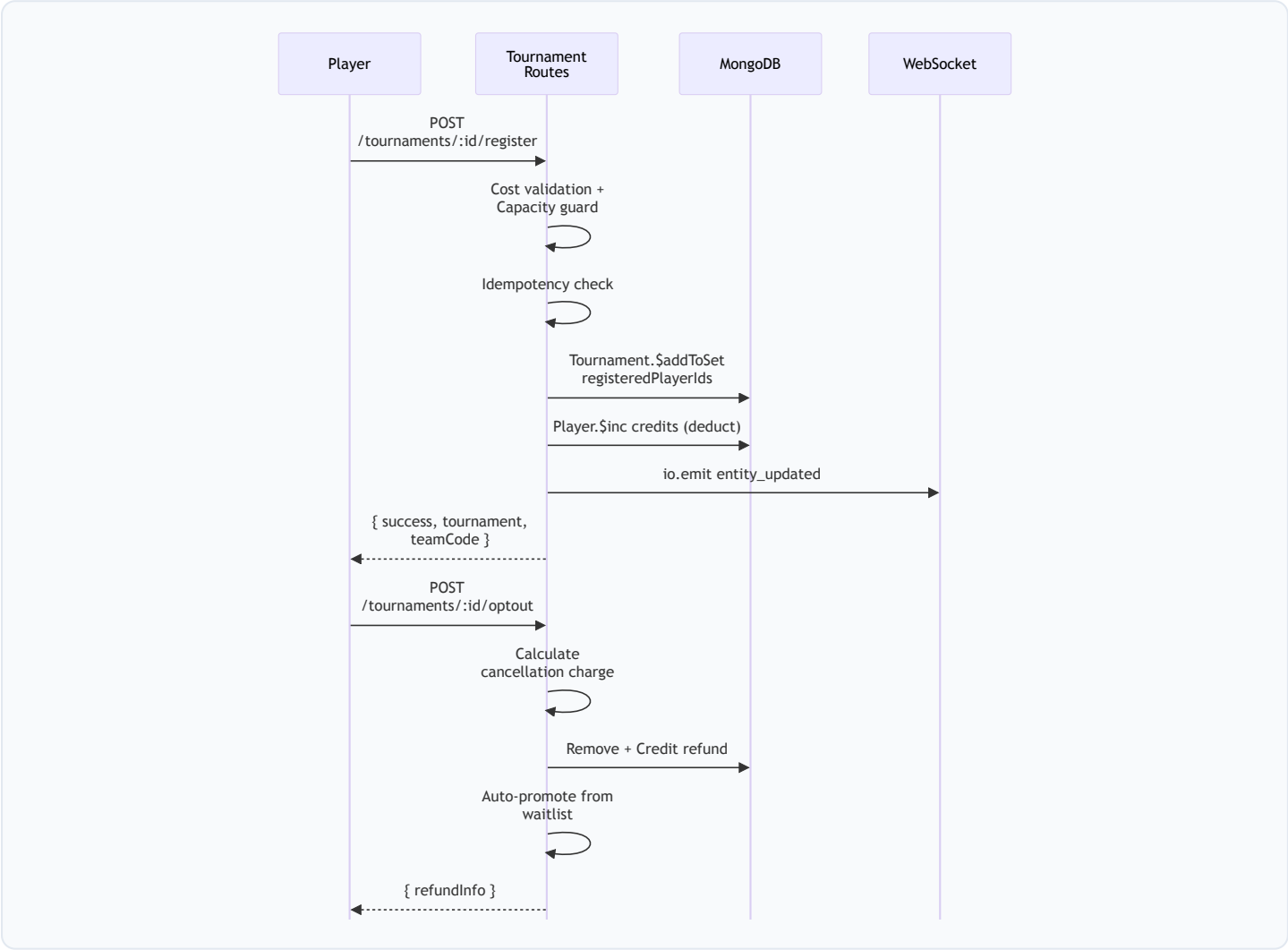
```
POST /api/v1/save { data, socketId, clientVersion }
```

API Endpoints

ENDPOINT	METHOD	AUTH	PURPOSE
/api/v1/data	GET	API Key + Cache	Full state hydration (role-filtered)
/api/v1/save	POST	API Key + Validate	Atomic state save with delta detection
/api/v1/status	GET	API Key	Check server version + latest app version

3. Tournament Lifecycle (Player Actions)

Players browse, register, manage partners, and check in to tournaments through a comprehensive registration pipeline with financial safeguards.



Detailed Flow Breakdown

- 1

Browse Tournaments
Player views available tournaments from the hydrated state data. Tournaments include title, sport, date, location, cost, registered count, and capacity. No API call needed — data comes from initial hydration.
- 2

Registration with Cost Validation
Server-side cost validation (VAPT-BL2 security fix) ensures the tournament cost matches the server's record, preventing client-side manipulation. An idempotency key prevents double-charges on retry.

POST /api/v1/tournaments/:id/register { method, idempotencyKey }

3 Credits Deduction

Credits are atomically deducted from the player's wallet using MongoDB \$inc operation. A wallet history entry is created for audit trail. If credits are insufficient, registration fails with a clear error.

4 Doubles Partner Flow

For doubles tournaments, the registering player can specify a partner. The system creates a doublesTeam with a unique teamCode (e.g., "A1B2C3D4"), deducts cost for both players, and notifies the partner via push notification.

```
POST /tournaments/:id/register { registeringPartnerId }
```

5 Join Team via Code

A partner joins an existing doubles team by entering the teamCode. The system validates the code, checks the team isn't full, and adds the player as player2.

```
POST /tournaments/:id/join-team { teamCode }
```

6 Opt-Out with Tiered Refund

Cancellation charges are calculated based on time before the event: 0% if ≥5 days, 25% if ≥3 days, 50% if ≥1 day, 100% if less. Refund goes to the original payer. If a waitlisted player exists, they're auto-promoted.

```
POST /tournaments/:id/optout { refundToWallet, optOutMode }
```

7 Waitlist & Check-In

If a tournament is full, players can join the waitlist. On event day, players check in via the app to confirm attendance.

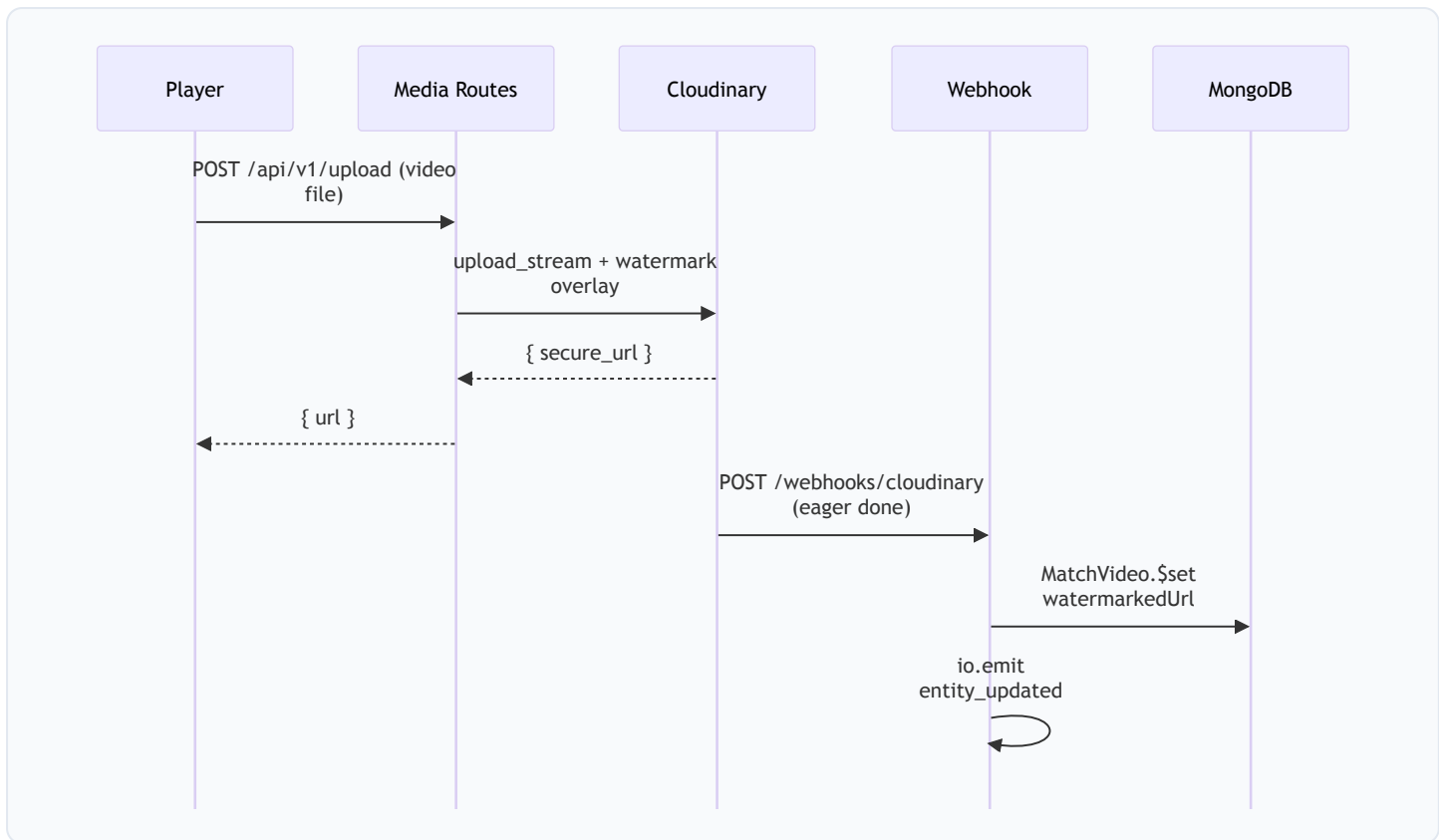
```
POST /tournaments/:id/waitlist | /check-in
```

API Endpoints

ENDPOINT	METHOD	PURPOSE
/tournaments/:id/register	POST	Register (credits/UIP/pending)
/tournaments/:id/optout	POST	Opt out with tiered refund
/tournaments/:id/waitlist	POST	Join waitlist for full tournament
/tournaments/:id/join-team	POST	Join doubles team with code
/tournaments/:id/partner-chat	GET/POST	Doubles partner messaging
/tournaments/:id/check-in	POST	Day-of-event check-in

4. Match Videos & Media

Players upload match videos which are processed with watermarks via Cloudinary's eager transformation pipeline, with webhook-based status updates.



Detailed Flow Breakdown

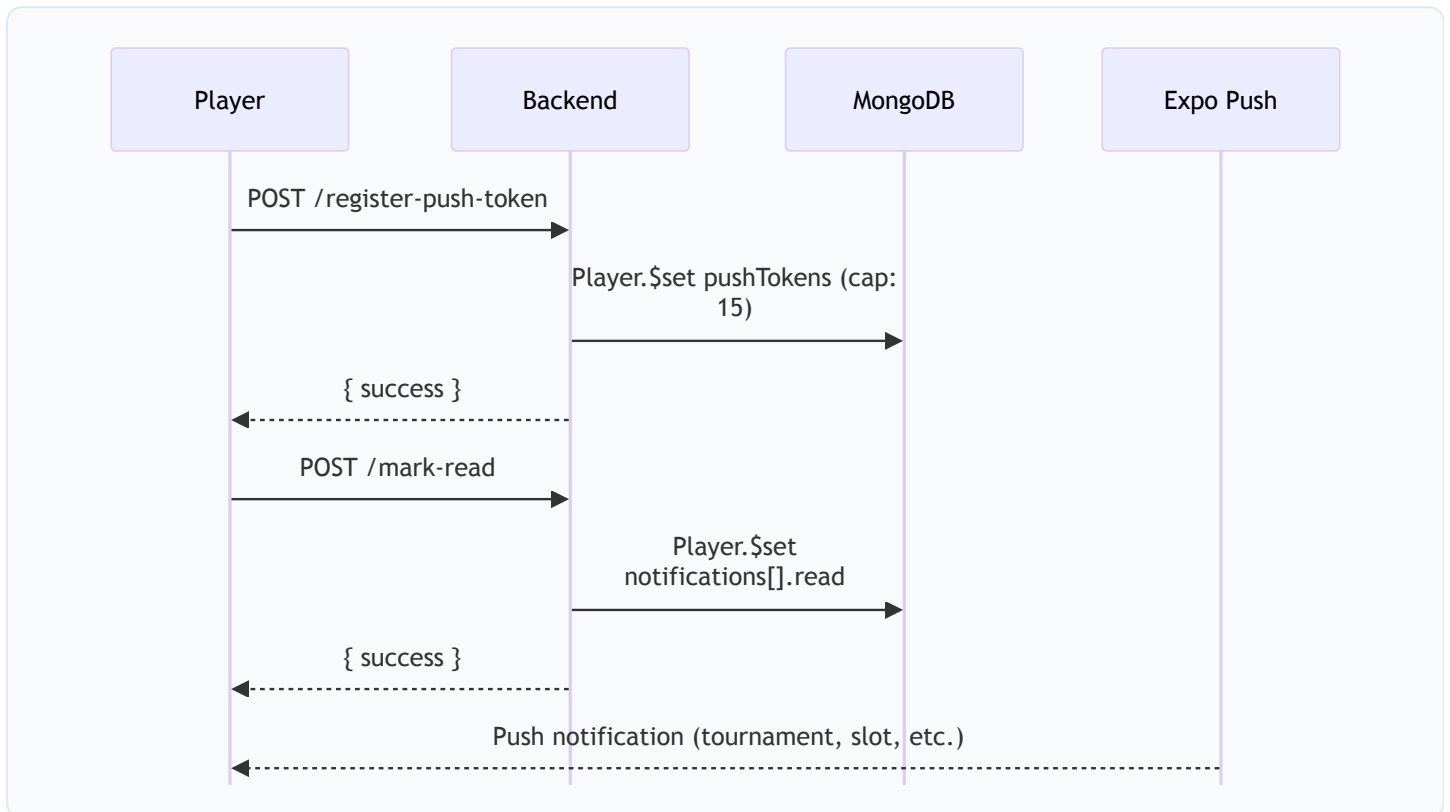
- 1 Video Upload**
 Player uploads a match video via multipart form data. The backend streams it to Cloudinary with an eager transformation that applies an AceTrack watermark overlay.
`POST /api/v1/upload (multipart/form-data)`
- 2 Watermark Processing**
 Cloudinary processes the video asynchronously. An eager transformation applies a diagonal watermark text across the video. The original (unwatermarked) URL is stored for purchased/downloaded versions.
- 3 Webhook Callback**
 When Cloudinary finishes the eager transform, it calls back to `/api/webhooks/cloudinary` with the watermarked URL. The server matches the video by `public_id` regex and updates the `MatchVideo` document.
`POST /api/webhooks/cloudinary (notification_type: eager)`
- 4 Metadata Save**
 The player saves video metadata (`matchId`, player associations, timestamps) via a separate call. The server creates or updates the `MatchVideo` document with `adminStatus: "Pending"` for review.
`POST /api/v1/videos/save-metadata { video: {...} }`
- 5 Real-Time Update**
 After the webhook updates the watermarked URL, the server emits an `entity_updated` event so all connected clients see the video's "ready" status in real-time.

API Endpoints

ENDPOINT	METHOD	PURPOSE
<code>/api/v1/upload</code>	POST	Upload video/image to Cloudinary
<code>/api/v1/videos/save-metadata</code>	POST	Save video metadata record
<code>/api/webhooks/cloudinary</code>	POST	Cloudinary eager transform callback

5. Notifications & Diagnostics

The notification system uses Expo Push for offline delivery and in-app notifications for real-time alerts. Diagnostics enable remote troubleshooting.



Detailed Flow Breakdown

- 1 Push Token Registration**
 On app launch, the player registers their Expo push token. The server caps stored tokens at 15 per user to prevent bloat from multiple devices. A security check ensures `userId` matches the authenticated user.
`POST /api/v1/register-push-token { userId, pushToken }`
- 2 Mark Notifications Read**
 Player marks one or all notifications as read. The server updates both the Player document and syncs to the legacy AppState for backward compatibility.
`POST /api/v1/mark-read { notifId }`
- 3 Push Notification Delivery**
 The server sends push notifications via Expo's push service for events like: tournament registration confirmation, slot opened from waitlist, coach assignment, partner opt-out alerts, and system updates.
- 4 Auto-Flush Diagnostics**
 The app periodically uploads diagnostic logs in the background. These are stored both locally and in Cloudinary, with a rotation policy keeping the last 3 files per user.
`POST /api/v1/diagnostics/auto-flush { username, deviceId, logs }`

5 Admin-Triggered Pull

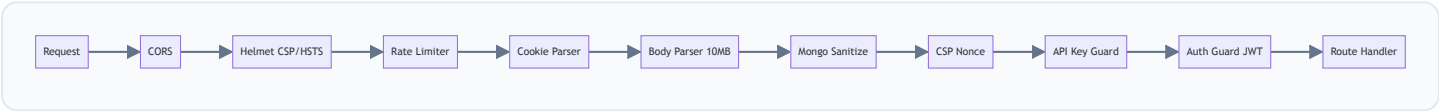
Admins can remotely request diagnostic logs from a specific device via WebSocket. The device receives a `force_upload_diagnostics` event and uploads its current log buffer.

API Endpoints

ENDPOINT	METHOD	PURPOSE
/api/v1/register-push-token	POST	Register Expo push token
/api/v1/mark-read	POST	Mark notifications as read
/api/v1/diagnostics	POST	Upload diagnostic logs
/api/v1/diagnostics/auto-flush	POST	Background auto-flush logs

6. Coach Booking & Security

Players can book private coaching sessions and view their booking history. Every request passes through a multi-layer security middleware stack.



Detailed Flow Breakdown

1 View Coach Availability

Player browses available coaches from the hydrated state. Each coach has a weekly availability schedule (day-of-week + time slots) that determines when they can be booked.

2 Create Booking

Player creates a booking request specifying the coach, date, time, and session type. The booking starts with status "pending" until the coach confirms.

```
POST /api/v1/bookings/create { coachId, date, time }
```

3 View My Bookings

Player can view all their past and upcoming bookings, including status (pending, confirmed, completed, rejected).

```
GET /api/v1/bookings/player/:playerId
```

4 CORS Protection

Cross-Origin Resource Sharing is configured with a whitelist of allowed origins. Requests from unknown origins are blocked.

5 Helmet Security Headers

Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), X-Frame-Options, and other security headers are applied to every response.

6 Rate Limiting

Global rate limiter prevents abuse. Specific endpoints (login, OTP, password reset) have additional strict rate limits to prevent brute force attacks.

7 API Key Guard

Every API request must include the `x-ace-api-key` header. This prevents unauthorized access from unknown clients and acts as a first line of defense.

8

JWT Auth Guard

Protected endpoints verify the JWT token from the HttpOnly cookie. The guard extracts userId and role for downstream authorization checks.

API Endpoints

ENDPOINT	METHOD	PURPOSE
/api/v1/bookings/create	POST	Create a new booking request
/api/v1/bookings/player/:playerId	GET	Get player's bookings